ThreadLocal

ThreadLocal产生的各种问题

一提ThreadLocal, 大家都知道每个线程都有一份变量副本, 各个线程相互隔离.

- 那ThreadLocal中的数据是怎么存的呢?
- 为什么key要做一个弱引用呢?
- 那value为啥不是弱引用呢?
- 既然key是弱引用, 那么gc之后, key一定会被回收吗, 一定会变成null吗?
- 如果key为null的话, 那ThreadLocalMap中会不会有大量key为null数据, 那怎么清理呢?
- ThreadLocal为啥容易内存泄漏呢?
- 怎么避免内存泄漏呢?

ThreadLocal中的数据是怎么存的呢?

首先, 每个线程内有一个ThreadLocalMap类型的变量, ThreadLocalMap其实就是一个HashMap, 它底层用一个Entry数组来存数据, Entry中的key是一个ThreadLocal弱引用, value就是资源.

调用set方法时, 其实就是以当前的ThreadLocal对象为key, 以资源为value放入ThreadLocalMap中. 调用get方法时, 就是以当前的ThreadLocal对象为key, 然后从ThreadLocalMap中取值.

开发中, 我们一般把ThreadLocal声明为static变量, 这个时候, 这个ThreadLocal对象就是所有线程共享的.

那么问题来了, key都共享了, 都是一样的, 那不同线程取出来的value不就是同一个?

- 不用担心, ThreadLocalMap是每个线程私有的, 每个线程的ThreadLocalMap都不一样.
- 所以即便相同的key, 从不同的ThreadLocalMap中取出来的值肯定不是同一个.

key弱引用

那为什么key要做一个弱引用呢?

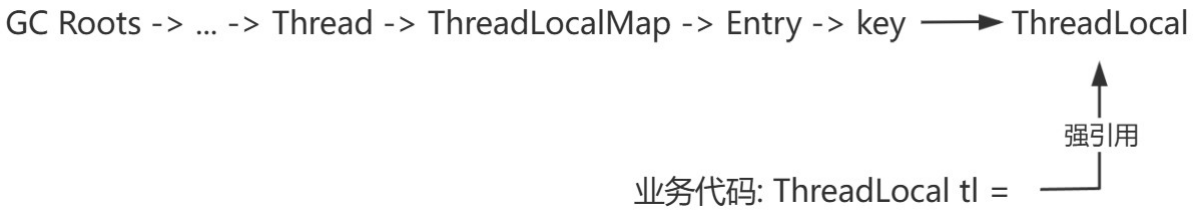
- 因为要避免key的内存泄漏.

那为啥会出现内存泄漏呢? 怎么平时也不经常说内存泄漏, 为啥一提ThreadLocal就说容易内存泄漏呢?

- 正常new一个对象, 基本都是局部变量, 方法执行完了, gc就把这些对象清理了.
- 如果你整一个全局的map或者list, 一直往里存东西, 也不清理, 那也会内存泄漏.
- 一般来说, 这种全局的map或者list, 都是本地缓存或者其他第三方库中会用到的, 内存泄漏的情况一般第三方库都处理好了, 不需要开发人员操心.

那ThreadLocal内存泄漏是什么原因呢, 怎么就需要开发人员操心了?

- 如果我们正常去new Thread(), 用完之后正常销毁, 那Thread类中的ThreadLocalMap这个属性也就被gc回收了, 不会出现内存泄漏.
- 但是, 项目中没人直接new Thread, 都是使用线程池. Springboot项目中处理请求也是从tomcat线程池中拿出线程然后处理请求, 线程池中的线程是不会销毁的
- 也就是说会一直存在一条引用链, 从Thread到ThreadLocalMap到Entry到key到ThreadLocal.



- 即便我们不想用ThreadLocal了, 将ThreadLocal对象设置为null. 那么这条引用链还是存在的, Entry对key的引用也是存在的.

- 这个key对应的ThreadLocal已经不用了, 但是一直无法被gc回收.
- 把这个key设置为弱引用, 如果它只被弱引用指向, 那么只要发生垃圾回收, 就能直接被回收. 从而避免内存泄漏.

那么问题来了, 既然key是弱引用, 那么gc之后, key一定会被回收吗, 一定会变成null吗?

- 不一定.
- 弱引用被gc回收的前提是, 这个对象只有弱引用关联, 没有其他引用.
- 如果ThreadLocal对象没有设置为null, 那就还有一个强引用指向它, 所以你得把ThreadLocal对象设置为null, 强引用才会断开, 就只剩了Entry的key的弱引用, gc才能自动回收.

那ThreadLocalMap会不会出现了大量key为null的数据?

- 不会.
- 当我们调用ThreadLocal的get, set, 或者remove方法时, 就会遍历Entry数组, 清理key为null的无效Entry.
- 然后会通过线性探测法重新处理hash冲突.

什么是线性探测法?

- 数据结构哈希表那一节, 处理hash冲突有很多种方法, Java的HashMap用的就是拉链法, 通过数组+链表实现.
- 而ThreadLocalMap这里虽然也是HashMap, 但是处理Hash冲突使用的方法是线性探测法.
- 当一个key要放入数组中时, 数组这个位置已经有数据了, 这就是发生了hash冲突, 那么就往数组的下一位放, 下一位也有了, 就往下下一位放, 这就是线性探测法.

value强引用

key是弱引用说完了, 那为啥value不是弱引用呢?

- 把数据存到value中, 是为了后面去用.
- 如果value是弱引用, 现在代码中可能还没有用value, 也就是说暂时没有强引用指向value, value只有弱引用, 那gc之后, 就错误地把value回收了.

- 但是value后面可能还要去获取去用呀. 所以value不能直接被gc回收, 它必须是强引用.

那么问题来了, value是强引用的情况下, 后续我们业务代码不用value了, 不去引用value, 也会一直存在着从Thread到ThreadLocalMap到Entry到value的引用链, gc无法回收, 导致value的内存泄漏, 那怎么办?

这里是强引用, 即便业务代码中不使用, gc也无法回收

GC Roots -> ... -> Thread -> ThreadLocalMap -> Entry -> value—强引用➡ Object资源

- 所以在使用ThreadLocal完后, 必须调用remove方法，手动断开value的强引用.

ThreadLocal和锁

ThreadLocal和锁都是为了解决并发安全问题, 那有啥不同呢?

- 锁的思想是共享资源只有一份, 强制多个线程必须互斥访问.
- ThreadLocal的思想是将这个资源复制多份, 每个线程一个副本, 各访问各的, 不冲突.

ThreadLocal这个思想也是处理死锁的一种思想. 如果你学习过操作系统, 那你就知道死锁有四个必要条件, 互斥条件, 不可剥夺条件, 请求保持条件, 循环等待条件.

- 互斥条件就是说共享资源必须互斥访问, 不能被同时访问. ThreadLocal的思想不就是让每个线程操作自己的数据副本, 一人一个, 大家各访问各的. 这不就破坏互斥条件了吗.
- 请求保持条件说的是, 多个线程各占有一份资源, 同时还在请求其他线程的资源, 而且每个线程都不愿放弃自己的资源, 从而互相等待, 形成死锁.

- 举个例子, 线程1和线程2需要获取两个锁A和B后才能继续执行。如果线程1持有锁A并尝试获取锁B，同时线程2持有锁B并尝试获取锁A，双方互相等待, 就可能死锁.
- 上一期视频聊过的, ReentrantLock有一个tryLock方法, 尝试加锁, 加锁失败就可以做其他操作. 通过这个就可以设计一种机制, 如果不能一次性获取所有需要的锁，则主动释放已持有的锁. 那线程1持有锁A尝试获取锁B, 使用tryLock发现获取失败, 然后释放锁A, 再次重试. 这就打破了请求保持条件.